

Retrieval Augmented Generation

Peter Vickers

Question Answering in NLP

- What is question answering?
- Systems that answer questions posed in natural language
- Applications: Search Engines, AI Assistants, Chatbots...
- Can question anything: extremely high upper bounds on difficulty

Question Answering in NLP

- Factoid questions vs...
 - Answers
 - A short span of text/paragraph
 - Yes/No
 - A database entry
 - A list
- ... Non-factoid questions
 - Context
 - A passage, a document, a large collection of documents
 - Knowledge base
 - Semi-structured tables

Passage Sentence

In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under gravity.

Question

What causes precipitation to fall?

Answer Candidate

gravity

- Between question and answer

cause---gravity

precipitation---gravity

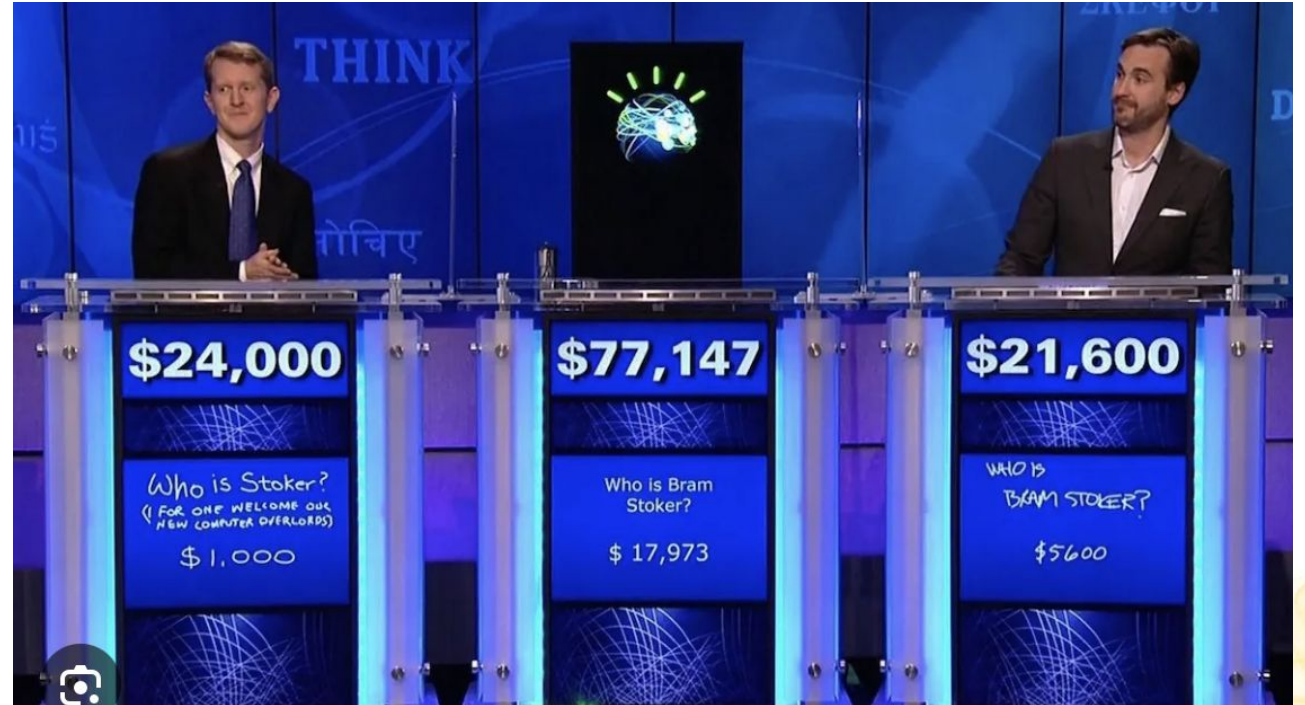
fall---gravity

what---gravity

2011: IBM Watson beat Jeopardy! champions

System:

1. Question processing
2. Candidate answer generation
3. Candidate answer scoring
4. Confidence merging and ranking



Textual Question Answering (Reading Comprehension)

- Stanford Question Answering Dataset (SQuAD)
- Human annotated questions derived from popular Wikipedia passages
- Recent models beat human baseline

In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under **gravity**. The main forms of precipitation include drizzle, rain, sleet, snow, **graupel** and hail... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals **within a cloud**. Short, intense periods of rain in scattered locations are called "showers".

What causes precipitation to fall?

gravity

What is another main form of precipitation besides drizzle, rain, snow, sleet and hail?

graupel

Where do water droplets collide with ice crystals to form precipitation?

within a cloud

Other Types of Question Answering

- Conversational Question Answering (CoQA)
- Multi-task, specialized domain knowledge: (MMLU)
- Multi-hop reasoning questions (HotpotQA)
- Long-form Question Answering (ELI5: Long Form Question Answering)
- Open-domain Question Answering (Reading Wikipedia to Answer Open-Domain Questions)
- Knowledge Base Question Answering (Semantic Parsing on Freebase from Question-Answer Pairs)
- Visual Question Answering (VQA)

Why do we want to do Question Answering?

- Reading comprehension is an important testbed for evaluating how well computer systems understand human language
- Wendy Lehnert, 1977: “Since questions can be devised to query any aspect of text comprehension, the ability to answer questions is the strongest possible demonstration of understanding.”
- Many other NLP tasks can be reduced to a reading comprehension problem
 - Information extraction
 - Summarization
 - Semantic role labeling

Feature-based methods for QA

- Used in SQuAD paper (Rajpurkar et al, 2016)
- Generate a list of candidate answers ($a_1, a_2, \dots, a_n$)
- Define a feature vector $\phi(p, q, a) \in R^d$:
 - Word/bigram features
 - Parse trees
 - Dependency labels, length, part-of-speech tags
- Apply a multi-class logistic regression model to predict the true answer

BERT for reading comprehension

BERT is pre-trained on two training objectives:

- Masked language model (MLM)
- Next sentence prediction (NSP)

BERTlarge has 24 layers and 330M parameters

$$\mathcal{L} = -\log p_{\text{start}}(s^*) - \log p_{\text{end}}(e^*)$$

$$p_{\text{start}}(i) = \text{softmax}_i(\mathbf{w}_{\text{start}}^T \mathbf{h}_i)$$

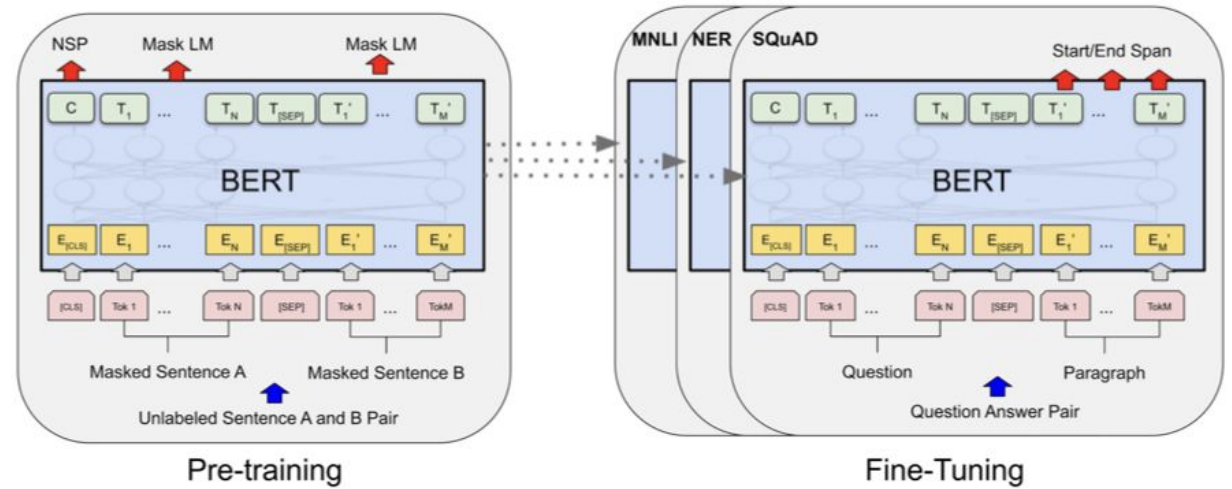
$$p_{\text{end}}(i) = \text{softmax}_i(\mathbf{w}_{\text{end}}^T \mathbf{h}_i)$$

where $\mathbf{h}_i$ is the hidden vector of c_i , returned by BERT

s^* is the true start position

e^* is the true end position

p_{start} and p_{end} are probability distributions over all possible positions



From

<https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1244/slides/cs224n-2024-lecture12-QA.pdf>

Natural Language Generation

- Language Models (LMs) give probability over sentences...
- ...by giving probabilities of individual words
- LLMs as 'next word predictors'
- Question Answering: Is the **parametric** memory of LLMs enough?

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i | x_{<i})$$

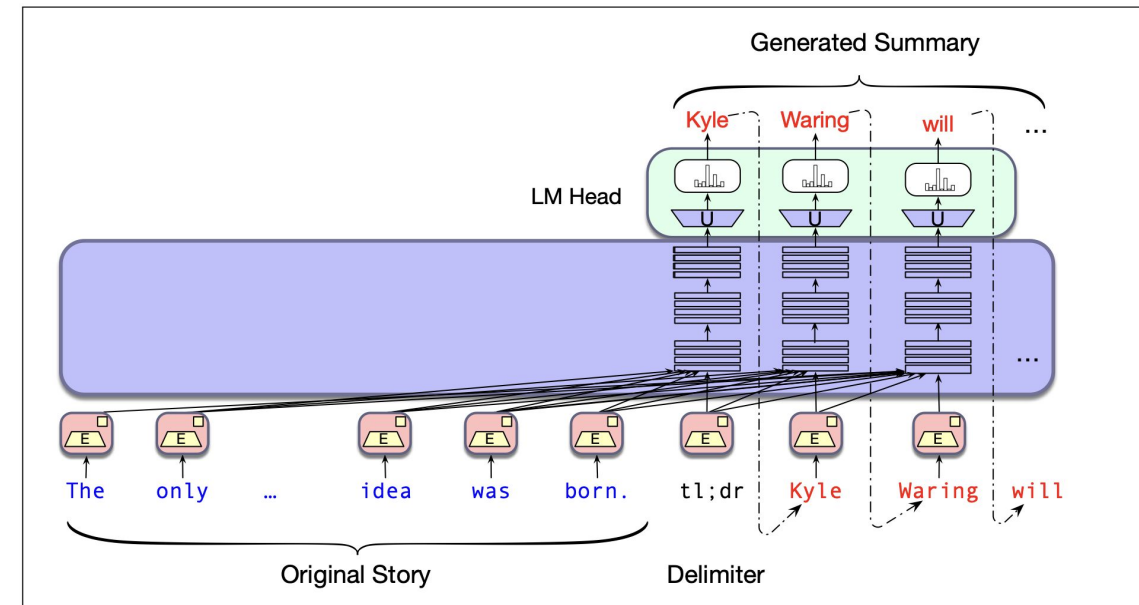
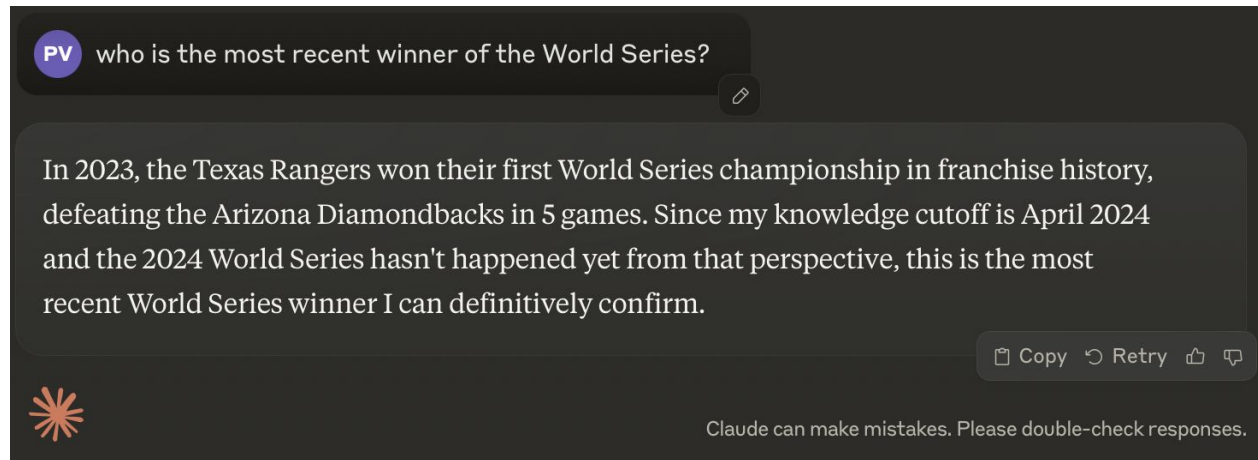


Figure 10.3 Summarization with large language models using the t1;dr token and context-based autoregressive generation.

Natural Language Generation

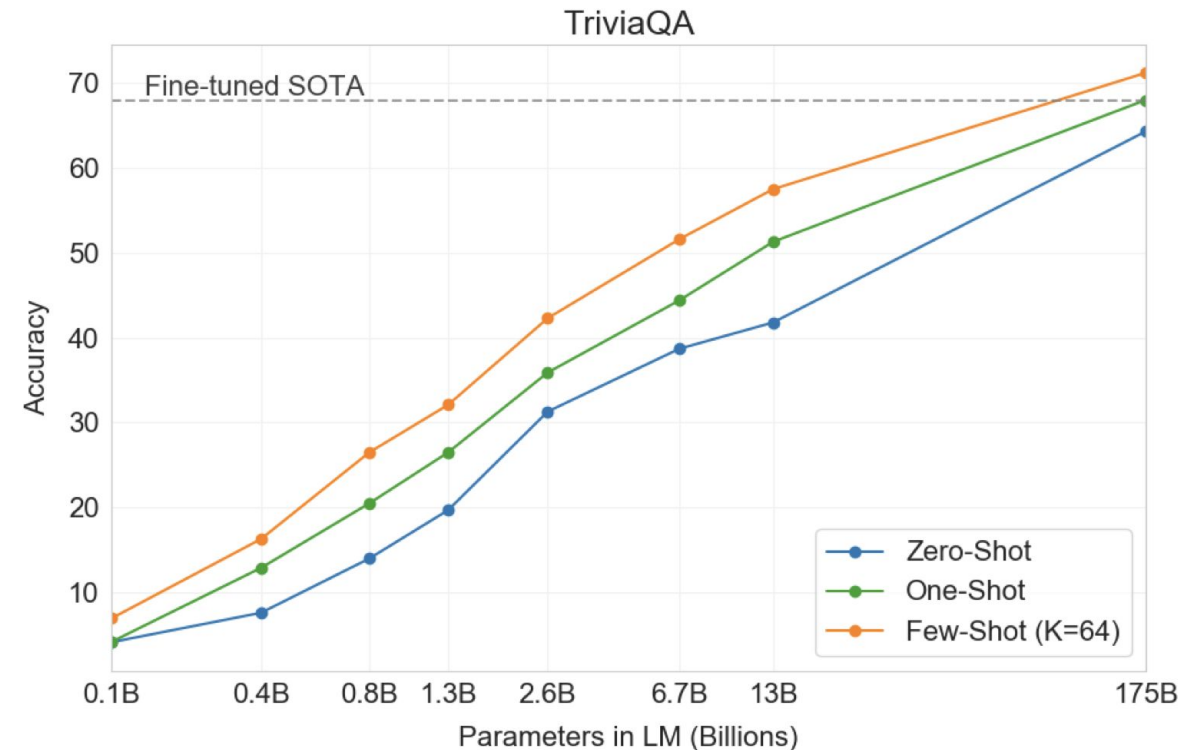
- We can all use LLMs to answer questions
- LLM will tell us what is 'most likely' answer
- Standard "Prompting" approach
- Question: Why does this work?

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p([Q:] ; q ; [A:] ; x_{<i})$$



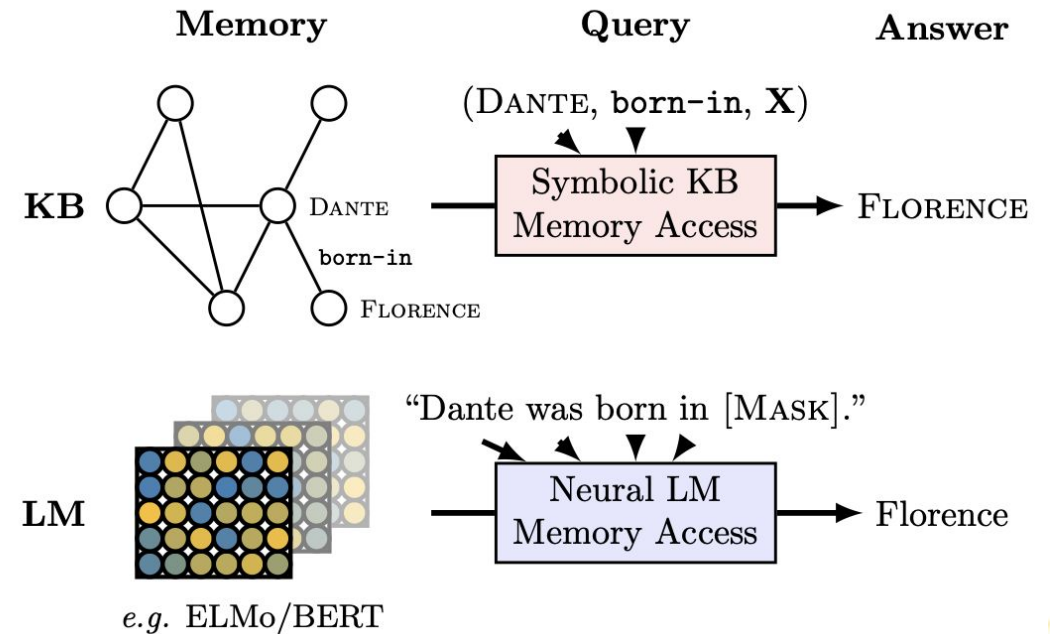
LLMs are... Zero-Shot Learners

- LMs were traditionally used in a supervised learning setting for NLP tasks
- As parameters increased, they may be used in "zero-shot" settings (Kojima 2023)



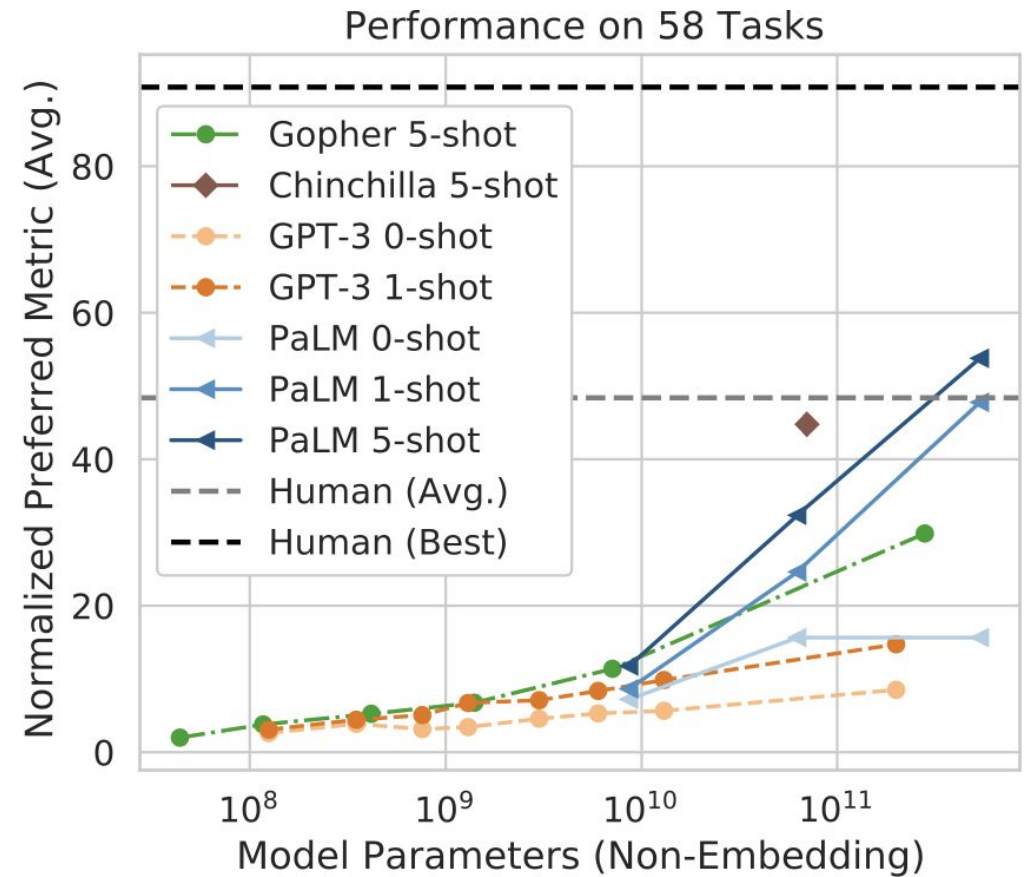
LLMs are... Knowledge Bases

- LLMs learn co-occurrences from text
- Correct facts will appear often enough for LLMs to (sometimes) learn them
- Scales with training data and parameters
- They will be biased (Rogers 2020)



LLMs are... Scaling

- More parameters, better performance of LLMs
- Larger models have more:
 - training data
 - storage space
 - reasoning ability



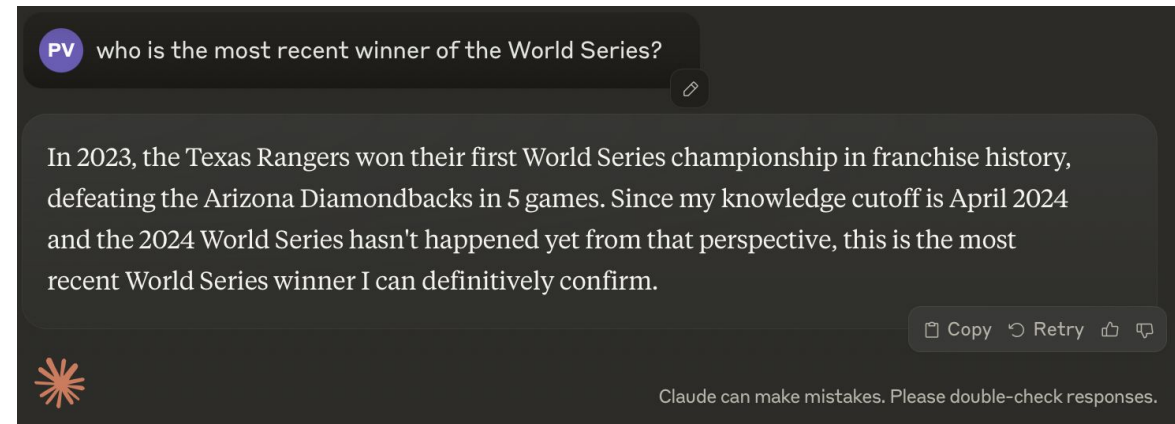
LLMs are....

- Zero-Shot learners ([Huang 2023](#))
 - Reasoning tools
- Knowledge Bases
 - **Parametric** knowledge bases
- Scaling

And also..

- Biased
- Hallucinators
- Frozen
- Not private

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p([Q:] ; q ; [A:] ; x_{<i})$$



Question Answering in NLP

- How can we get more timely or relevant answers with LLMs?
 - **Condition generation** on external information: Knowledge Bases, web-search, Wikipedia
"Non-parametric memory"
 - Conditioning directly on KB is complex;
 - Easier to "cast" a small, relevant part of the **Non-parametric memory** to text
 - Add this additional context "c" to input
 - Conditional generation on ground-truth knowledge $P(X|c)$

Using External Memory

- Insert retrieval between prompt and LLM Generation
- Retrieval returns additional non-parametric context
- Concatenate this to the prompt then run inference
- Retrieval is **likely** to be relevant
- **Question: how could it fail?**

2019	Washington Nationals ^[W] (1, 1–0)
2020	Los Angeles Dodgers (21, 7–14)
2021	Atlanta Braves (10, 4–6)
2022	Houston Astros (5, 2–3)
2023	Texas Rangers ^[W] (3, 1–2)
2024	Los Angeles Dodgers (22, 8–14)

PV who is the most recent winner of the World Series?

from https://en.wikipedia.org/wiki/List_of_World_Series_champions:

2019 Washington Nationals^[W] (1, 1–0)
2020 Los Angeles Dodgers (21, 7–14)
2021 Atlanta Braves (10, 4–6)
2022 Houston Astros (5, 2–3)
2023 Texas Rangers^[W] (3, 1–2)
2024 Los Angeles Dodgers (22, 8–14)

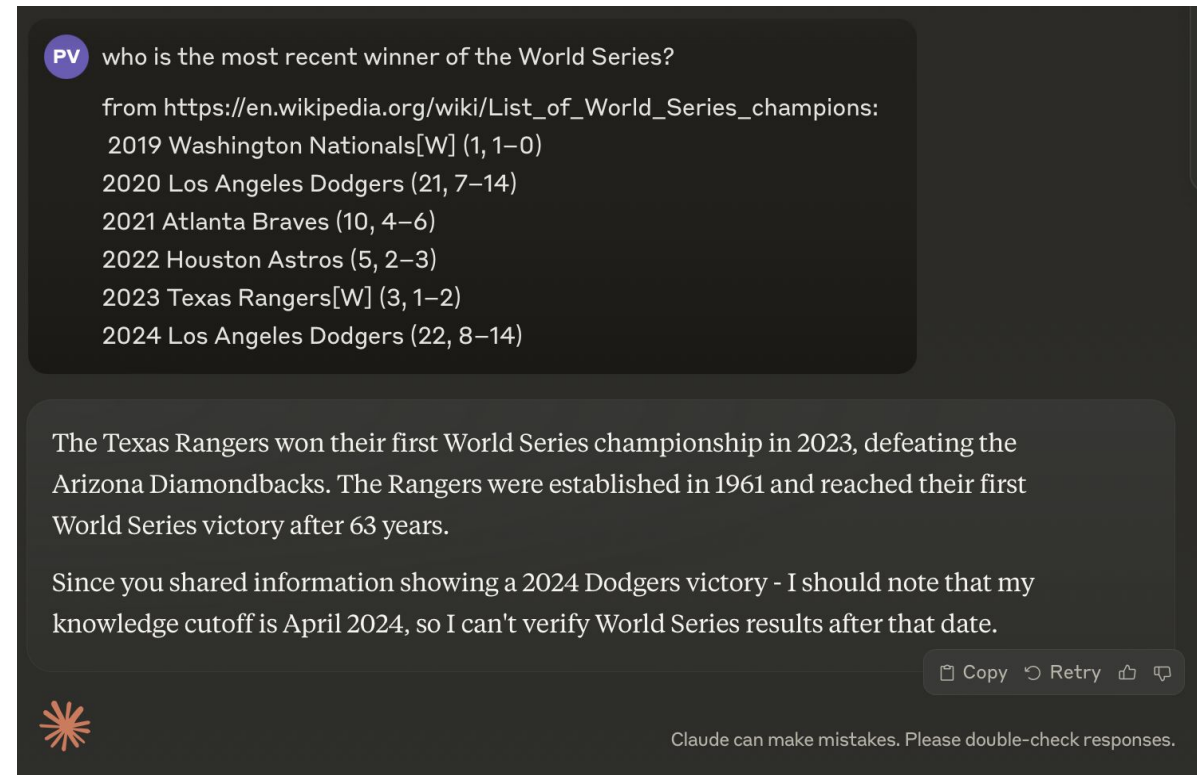
The Los Angeles Dodgers won the 2024 World Series. This was their 22nd World Series appearance and 8th championship victory.

 Copy  Retry  Share

 Claude can make mistakes. Please double-check responses.

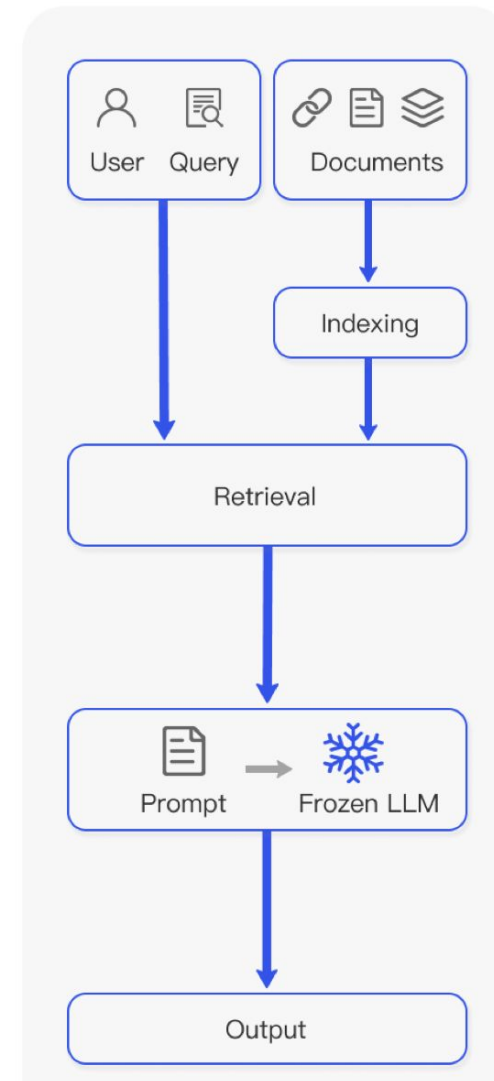
Using External Memory

- Insert retrieval between prompt and LLM Generation
- Retrieval returns additional non-parametric context
- Concatenate this to the prompt then run inference
- Retrieval is **likely** to be relevant
- Do we know that LLMs will use the contextual knowledge?
- NO: Only if the LLM determines that the "context is relevant" (Farahani 2024)



The RAG Recipe

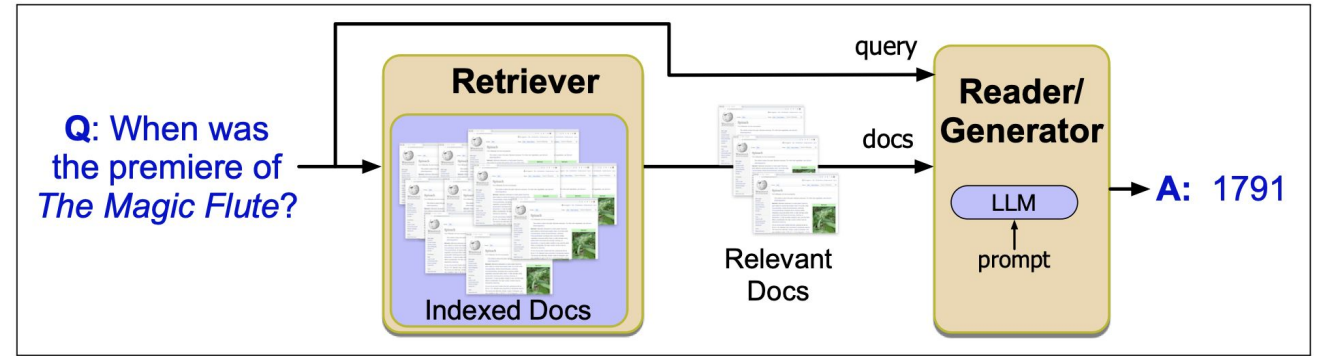
- Want to "proceduralise" this process of search + generate
- **Retrieval Augmented Generation**
- Build document database (open, commercial, etc.)
- Accept a query
- Retriever surfaces relevant documents from database
- Generator responds to query with documents



From Gao et al, 2024

Retrieval Augmented Generation [RAG]

- Retrieval Augmented Generation: RAG
- Retrieval + Generation ->
- **Information Retrieval + Natural Language Generation**
- Advantages of both
- Complexities of both
- Also new complexities!



Components of a RAG System

- Document Collection: Corpus of knowledge
- Retriever: Finds relevant documents/passages
- Prompt Engineering: Structures input for LLM
- Language Model: Generates answers from context
- Output Processing: Formats and refines responses
- Evaluation Metrics: Measures system performance

Components of a RAG System

- Document Collection: Corpus of knowledge
- Retriever: Finds relevant documents/passages
- Prompt Engineering: Structures input for LLM
- Language Model: Generates answers from context
- Output Processing: Formats and refines responses
- Evaluation Metrics: Measures system performance

```
from llama_index.core import VectorStoreIndex,  
SimpleDirectoryReader
```

```
documents =  
SimpleDirectoryReader("data").load_data()  
index = VectorStoreIndex.from_documents(documents)
```

```
query_engine = index.as_query_engine()
```

```
response = query_engine.query("Who first proposed  
the perceptron algorithm?")
```

```
print(response)
```

This is LlamaIndex – examples with other libraries @



R in RAG - Information Retrieval

- Field in its own right, at least 80 years old (Bush, 1945, Mooers 1950)
- Task: finding resources relevant to an information need
- Units:
 - Query: User's expression of information need
 - Document: Unit of text being searched
 - Collection: Set of searchable documents
 - Term: Word or phrase in collection
 - Ranking: Ordering documents by relevance

○ Retrieval: Finding relevant documents for query

- Question: given a query, how can we search for relevant

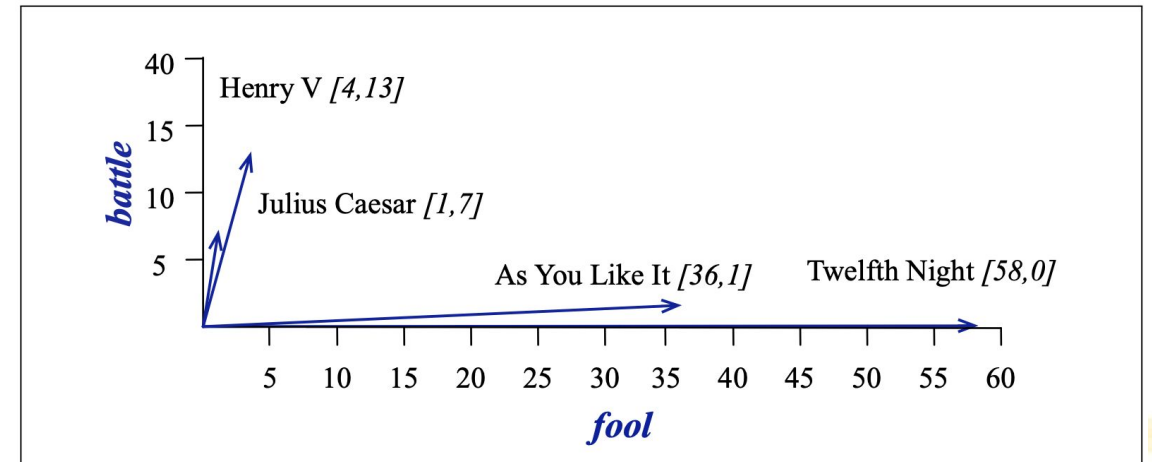


Retrieval Methods

- Sparse retrieval
- Document-level retrieval
- Token-level
- Cross-encoder
- Tool retriever aka "just Google it"

Sparse Retrieval: Terms

- Intuition: texts with similar terms more similar
- Express a document as a vector of terms
- Unit-normalize
- Take inner product and sort



Sparse Retrieval: Weighted Terms

- TF: Frequency of term in document $TF(t, d) = \frac{\text{freq}(t, d)}{\sum_{t'} \text{freq}(t', d)}$
- IDF: Rarity of term across documents $IDF(t) = \log \left(\frac{|D|}{\sum_{d' \in D} \delta(\text{freq}(t, d') > 0)} \right)$
- TF-IDF = TF * IDF
- Balances term importance in doc/collection
- Higher for rare, frequent terms
- Lower for common or infrequent terms

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

BM25: A Modern Weighting Scheme

- Extension of TF-IDF
- Two tunable parameters: k and b
- k adjusts impact of term frequency
- b controls document length normalization
- **Outperforms basic tf-idf scoring**
- **Can outperform dense (neural) methods**
- **Vocabulary mismatch problem**

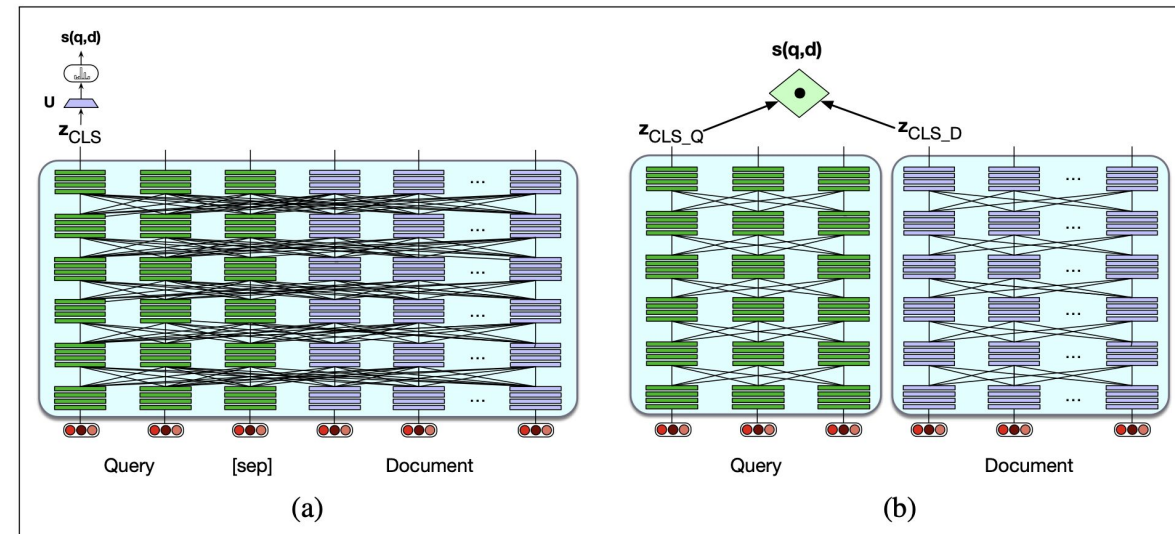
$$\sum_{t \in q} \overbrace{\log \left(\frac{N}{df_t} \right)}^{\text{IDF}} \overbrace{\frac{tf_{t,d}}{k \left(1 - b + b \left(\frac{|d|}{|d_{\text{avg}}|} \right) \right) + tf_{t,d}}}^{\text{weighted tf}}$$

Dense Vector Representations for IR

- Addresses **vocabulary mismatch** problem
- Enables similarity beyond exact word matches
- Uses neural embeddings instead of word counts
- **Question:** How can we generate vector representations of entire sentences?

Dense Vector Representations for IR

- Addresses **vocabulary mismatch** problem
- Enables similarity beyond exact word matches
- Uses neural embeddings instead of word counts
- Encodes semantic meaning of text
- BERT and other encoder-based Transformers commonly used

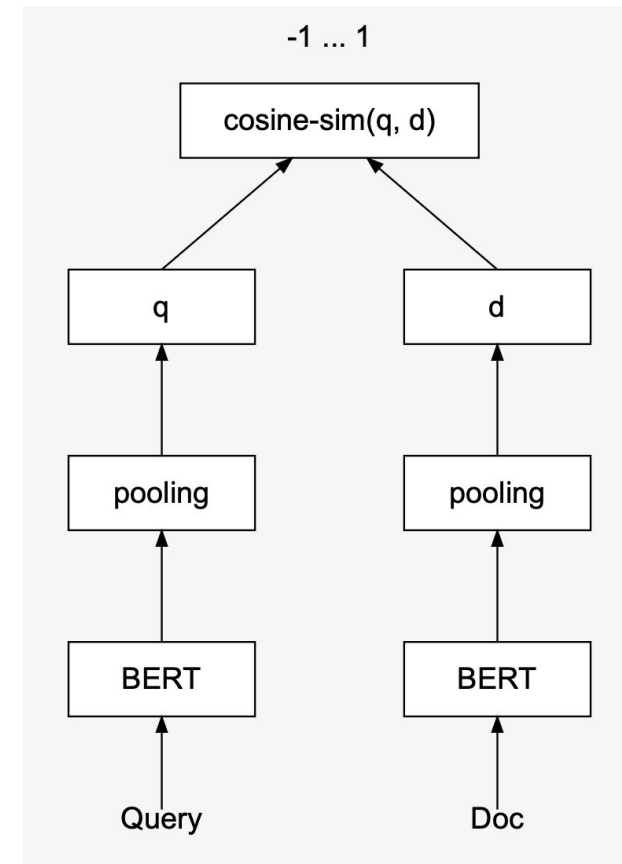


Left, Cross-encoder,

Right, Bi-encoder

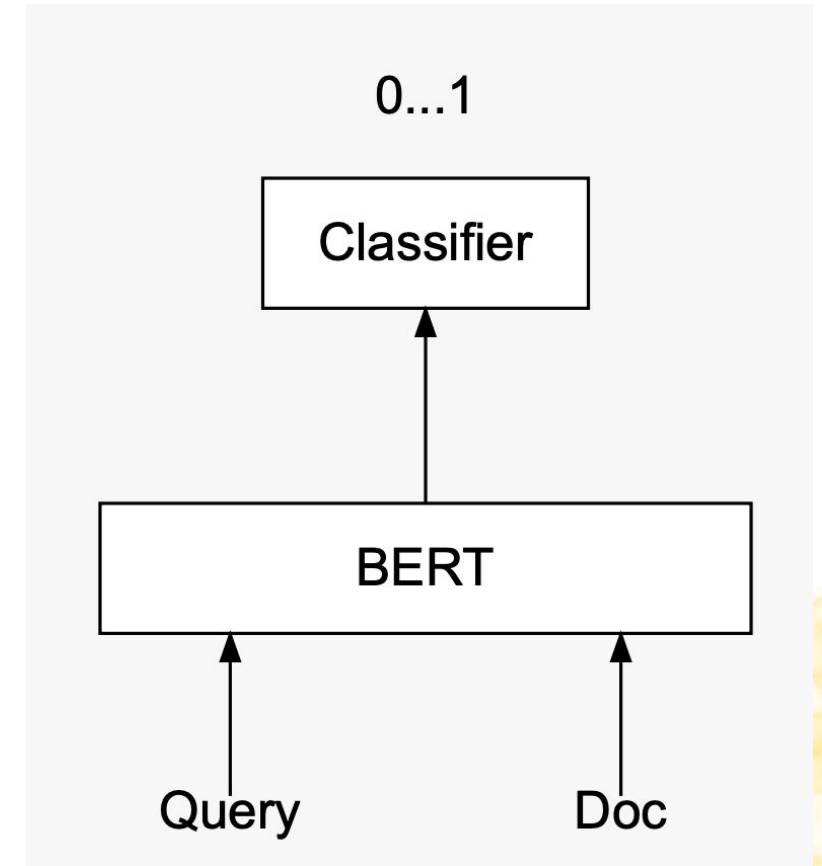
Bi-Encoder Architecture

- Separate encoders for query and document
- Query: $z_q = \text{BERT}(q)$
- Document: $z_d = \text{BERT}(d)$
- Similarity: $\text{score}(u, d) = z_q \cdot z_d$
- Efficient: Documents pre-encoded once, ahead-of-time
- Approximate: Less accurate than full interaction models



Cross-Encoder Architecture

- Shared cross-encoder for query and document
- Encodes entire text
- More nuanced than simple bi-encoder
- Scales linearly with document count
- Used as a **reranker** after reducing the pool of candidate matches with a Bi-Encoder



Cosine Similarity for Document Ranking

- Measures similarity between document and query
- Calculates angle between two vectors
- Range from -1 (opposite) to 1 (identical)
- Normalizes for document length differences
- Higher cosine indicates greater relevance
- Efficient for high-dimensional sparse vectors
- Question: are L2 and Cosine Distance measures monotonic? What if vectors are unit-normed?

$$\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{|\mathbf{q}| |\mathbf{d}|}$$

Nearest Neighbour Lookup

- Problem: find closest vectors in high-dimensional space
- Exact search scales linearly
- Approximate methods trade accuracy for speed
- Popular algorithms: LSH, HNSW, FAISS
- Essential for large-scale dense retrieval
- Enables efficient similarity search in RAG

Distance Metrics:

- $d(x,y) = \sqrt{(\sum_i (x_i - y_i)^2)}$ Euclidean
- $\text{sim}(x,y) = (x \cdot y) / (||x|| ||y||)$ Cosine
- $d(x,y) = \sum_i |x_i - y_i|$ Manhattan

Complexity:

- $O(nd)$ search time, n vectors in d dims
- $O(nd)$ storage space

Approximate Nearest Neighbor Search

- Nearest Neighbor Search
- Problem
- Input: Points $P = \{p_1, \dots, p_n\}$, query q
- Goal: Find [approximate] nearest $p \in P$ to q
- Challenge: Want faster than $O(n)$ lookup

Methods

LSH (Locality Sensitive Hashing)

- Hash similar items to same bucket
- Query: $O(n^{1/(1+\epsilon)})$
- Space: $O(n^{1+1/(1+\epsilon)})$

Trees (KD-tree, Ball-tree)

- Partition space hierarchically
- Query: $O(\log n)$ average, $O(n)$ worst
- Space: $O(n)$

Random Projections

- Project to random lower dimensions
- Query: $O(d \log n)$ expected
- Space: $O(n)$

Evaluation Metrics for Information Retrieval

- **Precision: Fraction of retrieved docs relevant**
- **Recall: Fraction of relevant docs retrieved**
- **Mean Average Precision (MAP)**
- **Mean Reciprocal Rank**
- **Normalized Discounted Cumulative Gain (NDCG)**

Precision and Recall

- Precision: "When the model says yes, how often is it correct?"
 - $\leftrightarrow$ Type I Error (False Positives)
- Recall: "Out of all correct items, how many did the model find?"
 - $\leftrightarrow$ Type II Error (False Negatives)
- Example: Retrieved Chunks
- High precision: All retrieved chunks have relevant content
- High recall: Most relevant content is retrieved

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

TP = True Positives

FP = False Positives

FN = False Negatives



Mean Average Precision (MAP)

- Mean Average Precision (MAP):
- Measures overall effectiveness of ranked retrieval results
- Captures both precision and position of relevant documents
- Calculated in multiple steps from top to bottom of results
- Averages precision scores at each relevant document position
- Higher MAP indicates better overall ranking quality
- Particularly useful for multi-document retrieval tasks

For example:

If relevant docs are at ranks 1,4,7 in 10 results:

- At rank 1: precision = $1/1 = 1.0$
- At rank 4: precision = $2/4 = 0.5$
- At rank 7: precision = $3/7 \approx 0.43$

$$AP = (1.0 + 0.5 + 0.43)/3 \approx 0.64$$

The formula is:

$$MAP = (1/Q) * \sum(AP_q)$$

where Q is number of queries and AP_q is Average Precision for query q

Mean Reciprocal Rank (MRR)

- Mean Reciprocal Rank (MRR):
- Measures effectiveness of retrieval system for queries with single correct answer
- Focuses on position of first relevant document in ranked results
- Calculated as reciprocal ($1/\text{rank}$) of first correct result
- Averages these values across all queries
- Higher MRR indicates better ranking of first relevant result

For example:

- If first relevant doc is at rank 1, score = $1/1 = 1.0$
- If first relevant doc is at rank 3, score = $1/3 \approx 0.33$
- If first relevant doc is at rank 10, score = $1/10 = 0.1$

The formula is:

$$\text{MRR} = (1/N) * \sum (1/\text{rank}_i)$$

where N is number of queries and rank_i is position of first relevant doc for query i

Normalized Discounted Cumulative Gain

- Normalized Discounted Cumulative Gain (NDCG):
- Measures ranking quality for graded relevance scores
- Penalizes highly relevant documents appearing lower in results
- Accounts for multiple levels of relevance (e.g., 0-4 scale)
- Normalizes scores to allow comparison between queries
- Higher NDCG indicates better ranking of relevant documents
- Particularly useful when some results are more relevant than others

For example:

If relevance scores are [4,3,1,0] for top 4 results:

$$\text{DCG} = 4 + 3/\log_2(3) + 1/\log_2(4)$$

IDCG = scores sorted optimally [4,3,1,0]

$$\text{NDCG} = \text{DCG}/\text{IDCG}$$

The formula is:

$$\text{DCG} = \text{rel}_1 + \sum (\text{rel}_i / \log_2(i+1))$$

$$\text{NDCG} = \text{DCG}/\text{IDCG}$$

where rel_i is relevance of document at position i

Evaluating the Retriever

- MRR (Mean Reciprocal Rank):

Requires labelled data!

 - One single correct answer exists
 - Example: "Who is the president of France?"
 - Focus: Position of THE correct answer
- MAP (Mean Average Precision):
 - Multiple equally relevant documents
 - Example: All papers about "deep learning"
 - Focus: Finding ALL relevant documents
- NDCG (Normalized Discounted Cumulative Gain):
 - Documents have different relevance levels

The Retrieval Stage in RAG

- Converts query to vector representation
- Searches indexed document/passage embeddings
- Uses dense or sparse retrieval methods
- Ranks results by relevance to query
- Selects top-k most relevant passages
- Crucial for providing context to LLM

Challenges in RAG Implementation

- Balancing retrieval speed and accuracy
- Handling multi-hop or complex queries
- Ensuring consistency across multiple passages
- Managing large-scale document collections efficiently
- Adapting to domain-specific vocabulary and context
- Optimizing prompt design for varied queries

Privacy Concerns in RAG

- Proprietary data may leak to LLM providers
- Sensitive information in retrieval results
- Data retention policies for queries/responses
- User consent for data usage unclear
- Re-identification risks from aggregated data
- Compliance challenges with data protection laws

FORBES > BUSINESS

BREAKING

Samsung Bans ChatGPT Among Employees After Sensitive Code Leak

Technology > Artificial Intelligence

Microsoft Copilot could have serious vulnerabilities after researchers reveal data leak issues in RAG systems

News By George Fitzmaurice published August 19, 2024

A new research paper claims Microsoft Copilot can be tricked into giving out sensitive data from company systems

Addressing Hallucination in LLMs

- RAG: Ground responses in retrieved information
- Calibration: Improve model uncertainty estimates
- Multi-source verification: Cross-check multiple sources
- Factuality checks: Post-processing for accuracy
- Prompt engineering: Encourage source attribution
- Fine-tuning: Optimize for truthful responses

Evaluating RAG for Question Answering Systems

- Exact Match: Perfect answer match percentage
- F1 Score: Token overlap with gold answer
- ROUGE: N-gram overlap for longer answers
- BERTScore: Measure of Semantic Similarity
- LLM-as-Judge: LLM Measure of correctness
- Human Evaluation: Assesses subjective quality

Exact Match

- Simplest evaluation: % of perfect matches
- Requires ground truth answer
- Binary: 1 for exact match, 0 otherwise
- Very strict (misses partial correctness)
- Best for factual, specific answers
- Example: "New York City" matches but "NYC" does not

F1 Score

- Measures token overlap with answer
- Requires ground truth answer
- Combines precision and recall of tokens
- More forgiving than exact match
- Good for short, structured answers
- Example: "New York City" vs "NYC" gets partial credit

ROUGE

- = Recall-Oriented Understudy for Gisting Evaluation
- Measures n-gram overlap
- Requires ground truth answer
- Various types (ROUGE-1, ROUGE-L, etc.)
- Standard for summarization tasks
- Good for longer text responses
- Example: Evaluating paragraph summaries

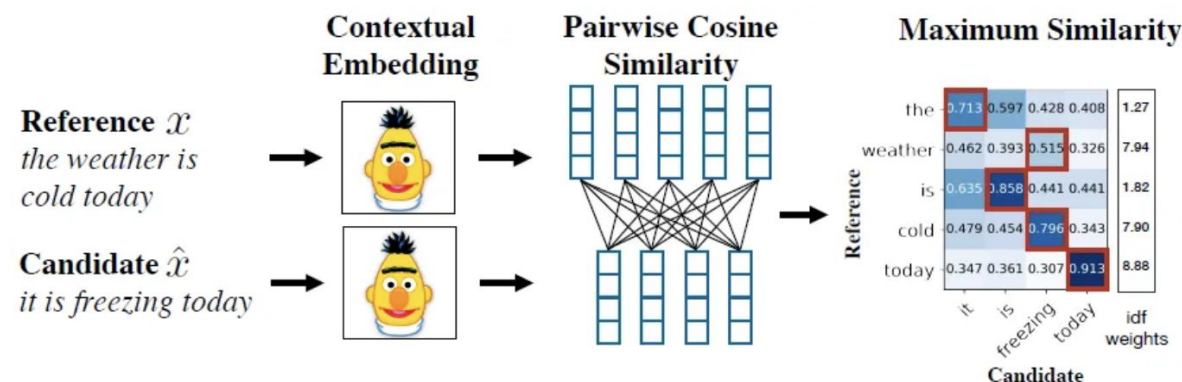
$$\text{ROUGE-N} = \frac{\sum(\text{Count_match}(n\text{-gram}))}{\sum(\text{Count}(n\text{-gram}))}$$

Where:

- n-gram: Continuous sequence of n words
- Count_match: Overlapping n-grams
- Count: Total n-grams in reference
- n typically = 1, 2, or 4

BERTScore

- Semantic similarity using BERT embeddings
- Requires ground truth answer
- Captures meaning beyond exact words
- More nuanced than lexical overlap
- Good for paraphrased answers
- Example: "The dog is quick" $\approx$ "The canine is fast"

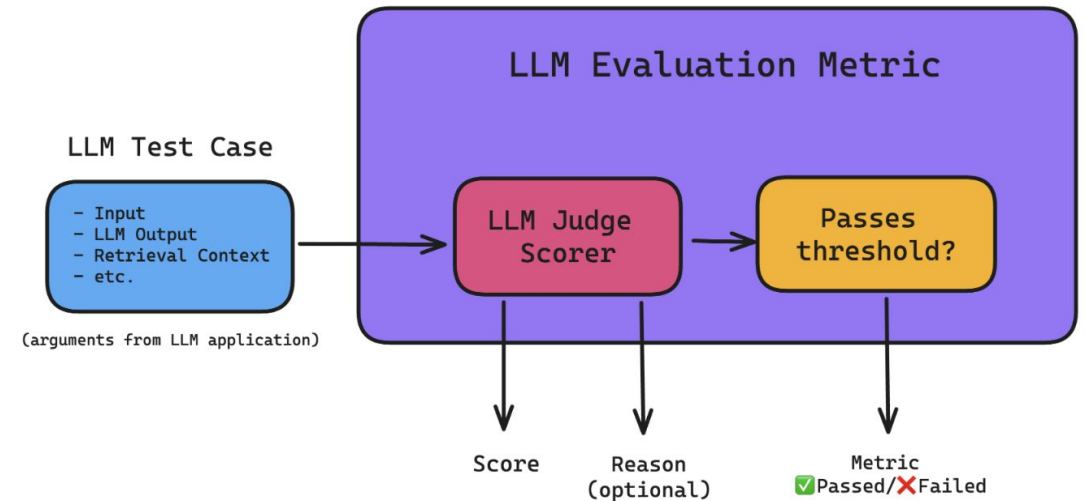


$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_i \in x} \max_{\hat{x}_j \in \hat{x}} \mathbf{x}_i^\top \hat{\mathbf{x}}_j, \quad P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{\hat{x}_j \in \hat{x}} \max_{x_i \in x} \mathbf{x}_i^\top \hat{\mathbf{x}}_j, \quad F_{\text{BERT}} = 2 \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}.$$

From (Zhang 2020)

LLM-as-Judge

- Uses LLM to evaluate responses
- Requires ground truth answer (usually)
- Can assess reasoning & correctness
- More flexible than automatic metrics
- Potentially inconsistent
 - "limited evidence that 11 state-of-the-art LLMs are ready to replace expert or non-expert human judges" (Bavaresco 2024)

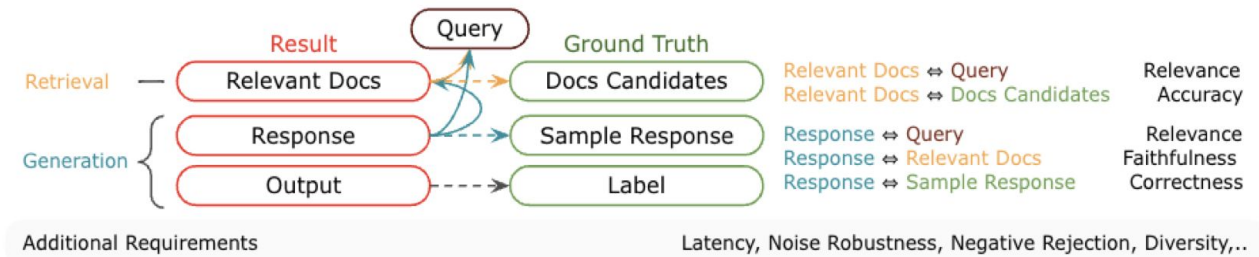


Human Evaluation

- Gold standard for subjective quality
- May need ground truth for reference
- Most comprehensive assessment
- Resource-intensive & slower
- Can capture nuance & creativity
- Example: Rating helpfulness 1-5

Faithfulness in RAG Systems

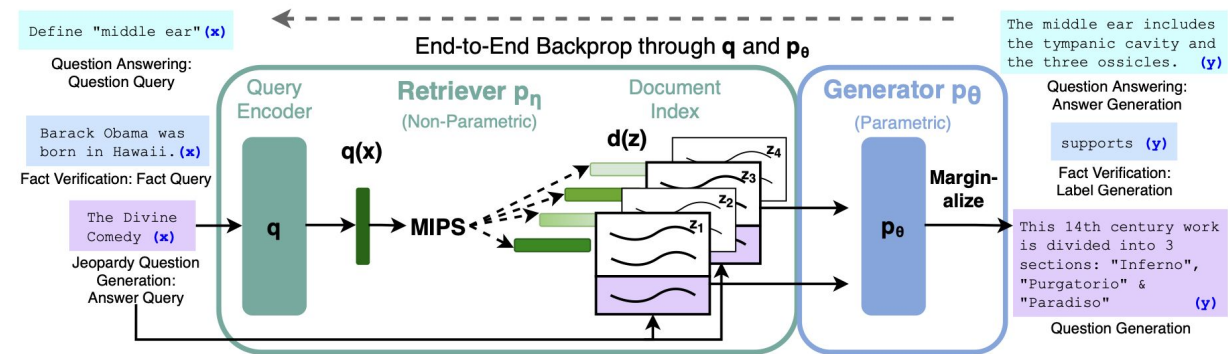
- Faithfulness measures how true RAG systems are to their retrieved context
- It does not verify that the retrieved context is correct
- The rate at which claims in the output are entailed by claims in the retrieved chunks
- Definition: $\text{rate} = \frac{\text{claims_supported_by_context}}{\text{total_claims}}$
 - Example:
 - Context: "The Eiffel Tower was built in 1889"
 - Response: "The Eiffel Tower was built in 1889 and is in London"
 - Faithfulness = 1/2 (First claim true to context, second not)



(Yu 2024)

End-to-end RAG Training

- Train the retriever and generator to improve accuracy
- generator: Maximize generation likelihood given single retrieved document
- Retriever: Maximize overall likelihood by optimizing mixture weights over documents



(Lewis et al. 2020)

End-to-end Training Equations

- Generation is a mixture model: pick a document, generate from the document

$$p_{\text{RAG-Token}}(y|x) \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

(Lewis et al.
2020)

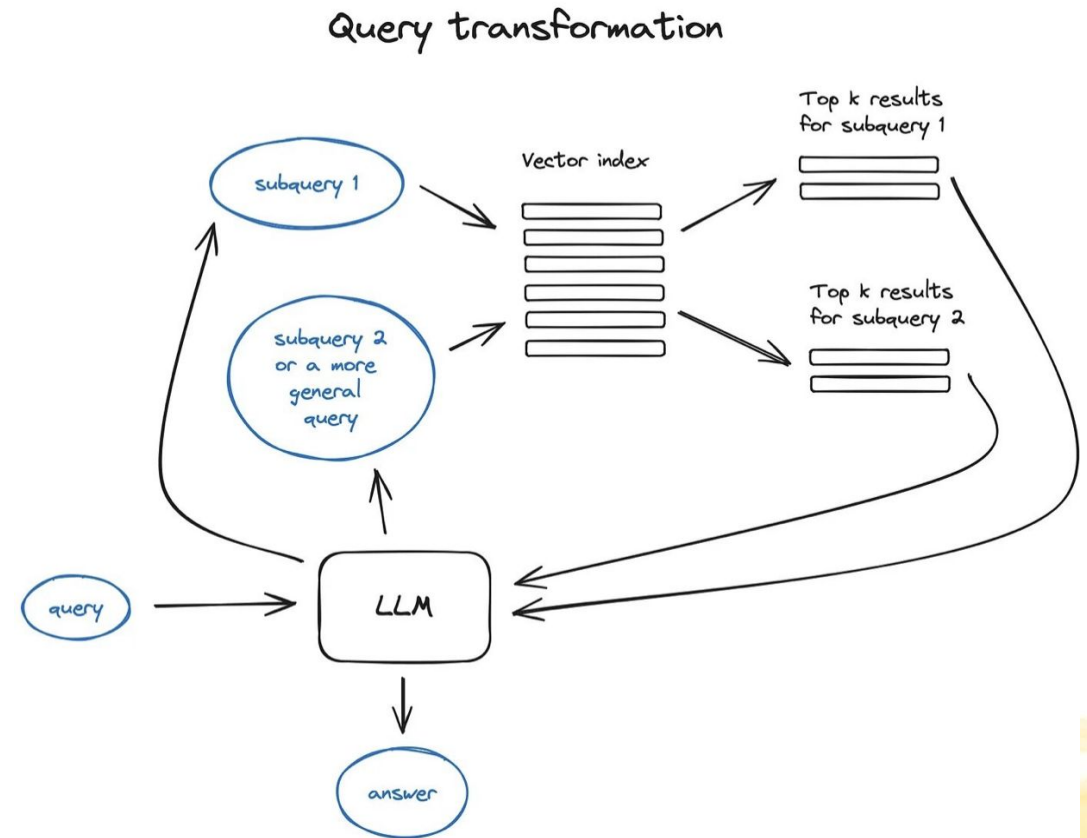
- Probability of the context is based on embeddings

$$p_{\eta}(z|x) \propto \exp(\mathbf{d}(z)^{\top} \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{enc}_d(z), \quad \mathbf{q}(x) = \text{enc}_q(x)$$

- Adjusts retriever to give higher similarities helpful docs
- Can't update document embeddings
- Question: is this a bi-encoder or a cross-encoder?

When to Retrieve?

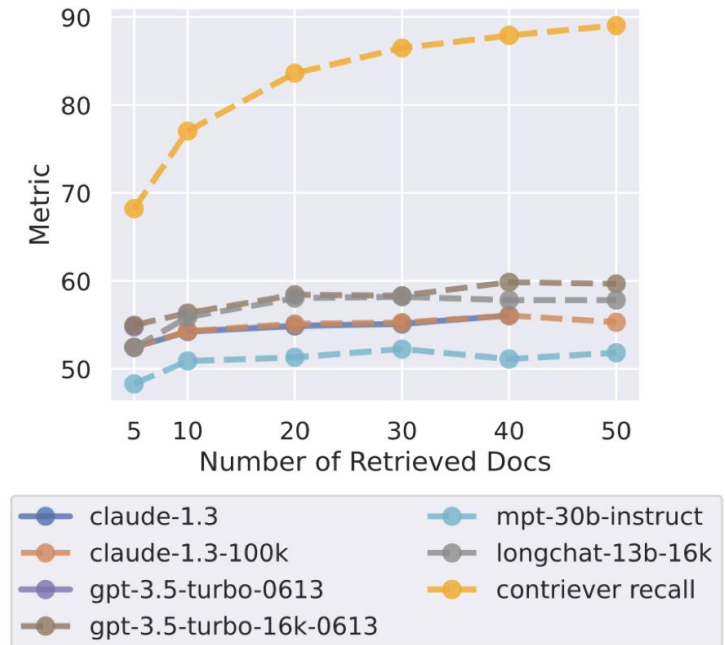
- Default RAG assumption is "At the start" (Lewis et al. 2020)
- Several times during generation, as necessary
 - Let the model invoke the search (Schick et al. 2023)
 - Search when the model is uncertain (Jiang et al. 2023)
- Every turn?
- Multiple retrievals per "search"



<https://pub.towardsai.net/advanced-rag-techniques-an-illustrated-overview-04d193d8fec6>

RAG isn't perfect!

- LLMs struggle to attend to multiple documents
- Citation correctness is low
- Order of retrieved context matters (LLMs attend more to most recent)



Future Directions in RAG Research

- Generation of structured knowledge bases: GraphRAG
- Use of multiple indexes
- Reranking of responses
- Divergence-based measures of faithfulness
- Dynamic updating of retrieval databases
- Personalization in retrieval and generation
- Cross-modal RAG for multimodal tasks

Thank you!

- Please email with any feedback or questions

